Experiment no 6

Aim : Apply K-Means and Hierarchical Clustering on sample datasets.

Dataset Used:
Dataset: Online Retail Customer Segmentation Dataset
Dataset Attributes:

- CustomerID
- Recency
- Frequency
- Monetary value

Theory
Clustering
Clustering is an **unsupervised machine learning technique** used to group similar data points together based on feature similarity.

K-Means Clustering :

K-Means partitions data into K clusters by minimizing within-cluster variance. Distance formula (Euclidean Distance):

$$d(x, c) = \sqrt{\sum_{i=1}^{n} (x_i - c_i)^2}$$

Objective Function:

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} ||x_i - \mu_k||^2$$

Where:

- C_k = Cluster k
- μ_k = Centroid of cluster k
- x_i = Data point

The goal is to minimize total intra-cluster distance.

Hierarchical Clustering :

Agglomerative clustering starts with each data point as a separate cluster and merges clusters iteratively. Ward's method minimizes variance between merged clusters.

Distance between clusters is calculated using linkage criteria.

$$S = \frac{b - a}{\max(a, b)}$$

● a = Average intra-cluster distance

● b = Average nearest-cluster distance Range:

● +1 → Strong clustering

● 0 → Overlapping clusters

● -1 → Poor clustering

Step-by-Step Procedure with Output
PART A: K-Means Clustering

CODE:

```
# ================================
# STEP 1 : Import Libraries
# ================================
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.cluster import AgglomerativeClustering

from scipy.cluster.hierarchy import dendrogram, linkage

import datetime as dt
print("Libraries imported successfully")
# ================================
# STEP 2 : Load Dataset
# ================================

df = pd.read_csv("OnlineRetail.csv", encoding='ISO-8859-1')

print("Dataset Loaded Successfully")
print(df.head())

# ================================
# STEP 3 : Data Cleaning
# ================================

# Remove missing CustomerID
df = df.dropna(subset=['CustomerID'])

# Remove negative or zero values
df = df[df['Quantity'] > 0]
df = df[df['UnitPrice'] > 0]

print("Data cleaned")
print(df.shape)

# ================================
# STEP 4 : Create Total Price
# ================================

df['TotalPrice'] = df['Quantity'] * df['UnitPrice']
```

VAIDEHI SETIYA D15B/49

```
print("TotalPrice column created")
print(df.head())
```

```
# ================================
# STEP 5 : Convert Invoice Date
# ================================

df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'], dayfirst=True)

print("Date converted successfully")

# ================================
# STEP 6 : Create RFM Features
# ================================

snapshot_date = df['InvoiceDate'].max() + dt.timedelta(days=1)

rfm = df.groupby('CustomerID').agg({
 'InvoiceDate': lambda x: (snapshot_date - x.max()).days,
 'InvoiceNo': 'count',
 'TotalPrice': 'sum'
})

rfm.columns = ['Recency','Frequency','Monetary']

print("RFM Table Created")
print(rfm.head())

# ================================
# STEP 7 : Feature Scaling
# ================================

scaler = StandardScaler()

rfm_scaled = scaler.fit_transform(rfm)

print("Feature scaling completed")
```

VAIDEHI SETIYA D15B/49

```
# ================================
# PART A : K-MEANS CLUSTERING
# ================================

# ================================
```

```
# STEP 8 : Elbow Method
# ================================

wcss = []
for i in range(1,11):
 kmeans = KMeans(n_clusters=i, random_state=42)
 kmeans.fit(rfm_scaled)
 wcss.append(kmeans.inertia_)

plt.figure(figsize=(8,5))
plt.plot(range(1,11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.show()

# ================================
# STEP 9 : Apply K-Means
# ================================

kmeans = KMeans(n_clusters=4, random_state=42)
rfm['KMeans_Cluster'] = kmeans.fit_predict(rfm_scaled)
print("KMeans clustering applied")
print(rfm.head())
# ================================
# STEP 10 : Visualize K-Means Clusters
# ================================

plt.figure(figsize=(8,6))

plt.scatter(
 rfm_scaled[:,0],
 rfm_scaled[:,1],
 c=rfm['KMeans_Cluster'];

 cmap='viridis'
)

plt.title("Customer Segments using K-Means")
plt.xlabel("Recency")
plt.ylabel("Frequency")
```

```
plt.show()
```

PART B: HIERARCHICAL CLUSTERING

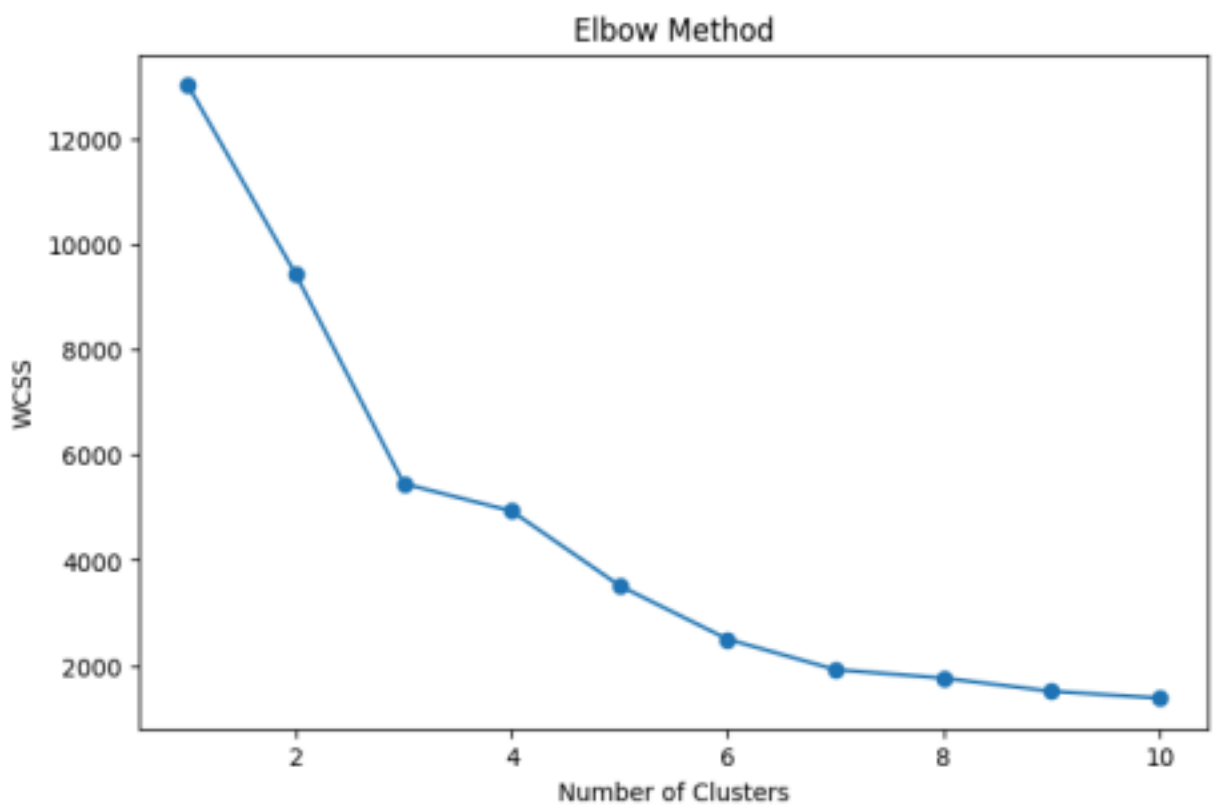
```
# ================================
# PART B : HIERARCHICAL CLUSTERING
# ================================

# ================================
# STEP 11 : Create Dendrogram
# ================================

linked = linkage(rfm_scaled, method='ward')

plt.figure(figsize=(10,6))

dendrogram(linked)

plt.title("Dendrogram for Hierarchical Clustering")
plt.xlabel("Customers")
plt.ylabel("Euclidean Distance")

plt.show()

# ================================
# STEP 12 : Apply Agglomerative Clustering
# ================================

hc = AgglomerativeClustering(n_clusters=4, linkage='ward')

rfm['Hierarchical_Cluster'] = hc.fit_predict(rfm_scaled)

print("Hierarchical clustering applied")
print(rfm.head())

# ================================
# STEP 13 : Visualize Hierarchical Clusters
# ================================

plt.figure(figsize=(8,6))
```

```
plt.scatter(
 rfm_scaled[:,0],
 rfm_scaled[:,1],
 c=rfm['Hierarchical_Cluster'],
 cmap='rainbow'
)

plt.title("Customer Segments using Hierarchical Clustering")
plt.xlabel("Recency")
plt.ylabel("Frequency")

plt.show()

# ================================
# STEP 14 : Cluster Summary
# ================================

cluster_summary = rfm.groupby('KMeans_Cluster').mean()

print("Cluster Summary")
print(cluster_summary)
```

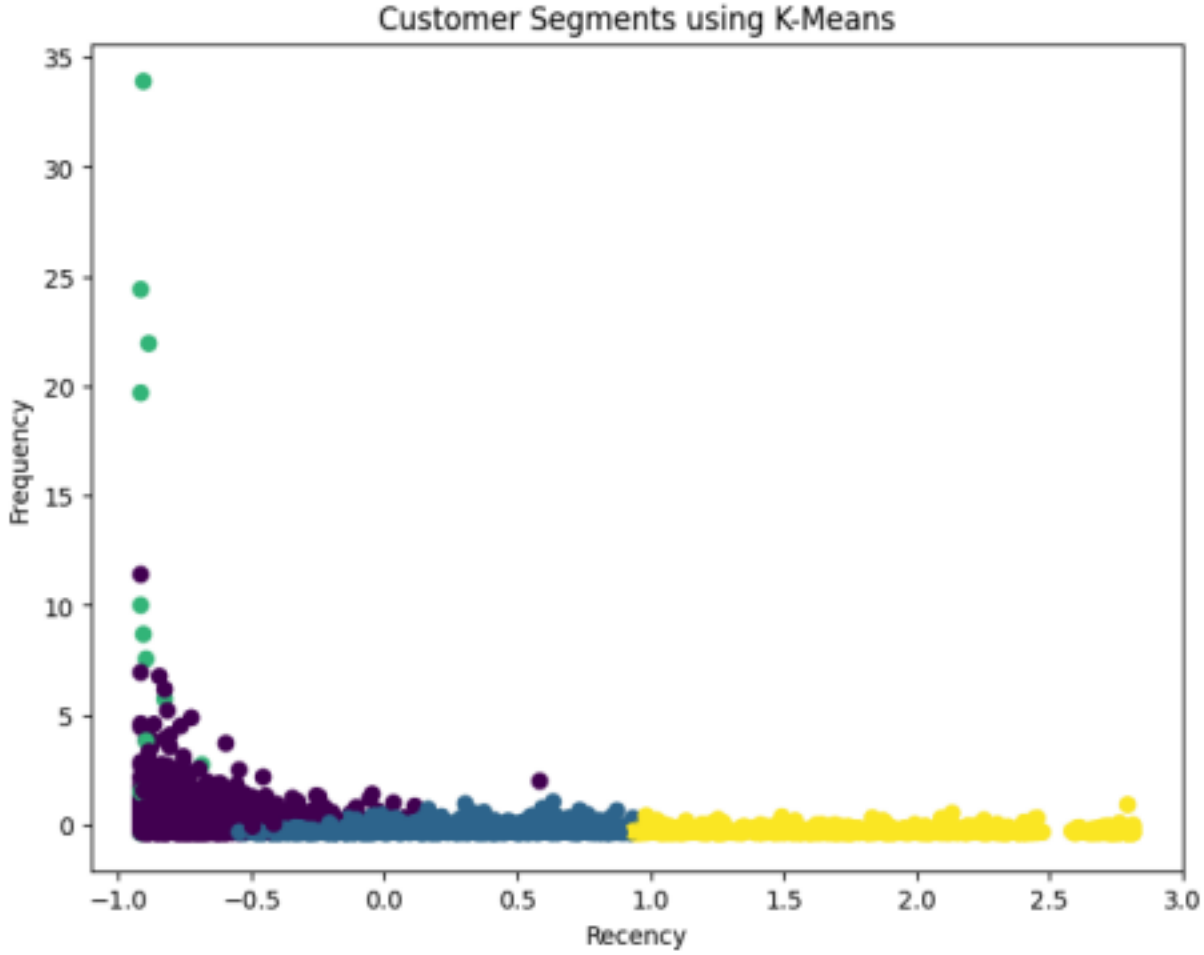
OUTPUT:

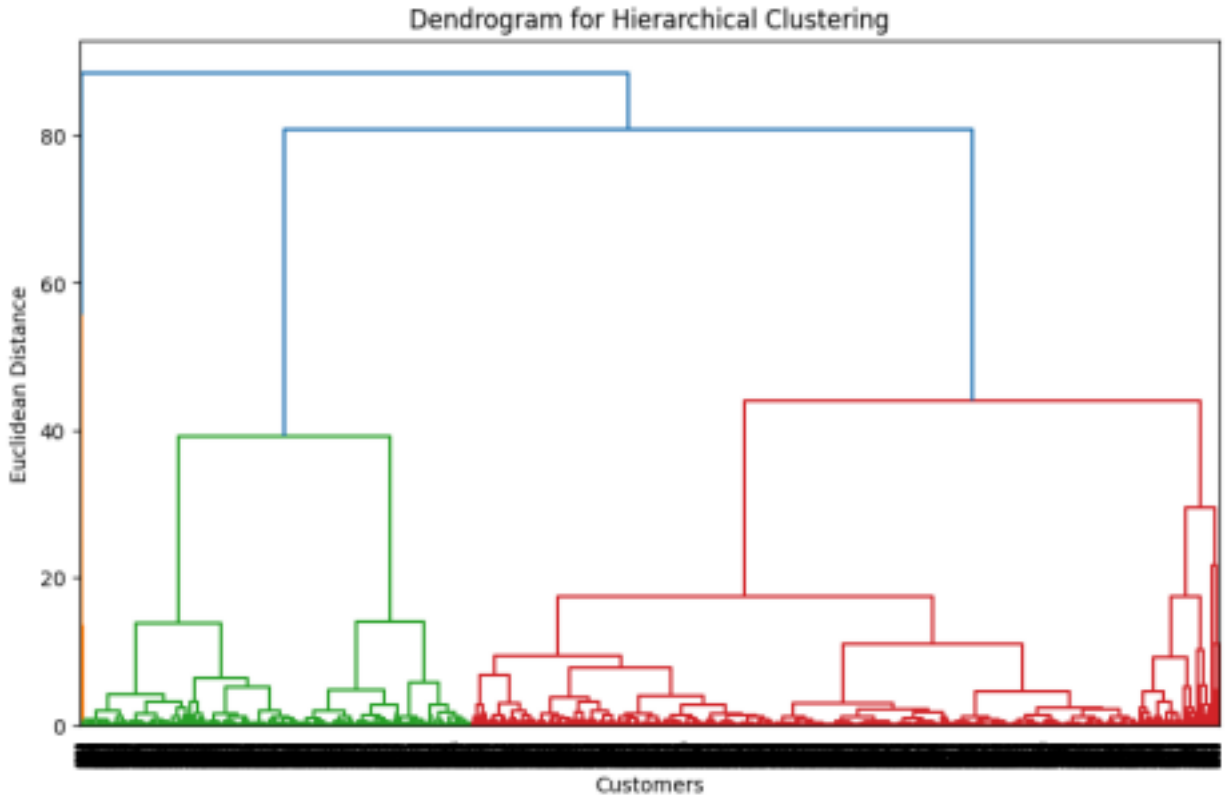
```
... Libraries imported successfully
    Dataset Loaded Successfully
      InvoiceNo StockCode                          Description  Quantity  \
    0    536365    85123A   WHITE HANGING HEART T-LIGHT HOLDER         6
    1    536365     71053                  WHITE METAL LANTERN         6
    2    536365    84406B       CREAM CUPID HEARTS COAT HANGER         8
    3    536365    84029G  KNITTED UNION FLAG HOT WATER BOTTLE         6
    4    536365    84029E       RED WOOLLY HOTTIE WHITE HEART.         6

            InvoiceDate  UnitPrice  CustomerID         Country
    0  01-12-2010 08:26       2.55     17850.0  United Kingdom
    1  01-12-2010 08:26       3.39     17850.0  United Kingdom
    2  01-12-2010 08:26       2.75     17850.0  United Kingdom
    3  01-12-2010 08:26       3.39     17850.0  United Kingdom
    4  01-12-2010 08:26       3.39     17850.0  United Kingdom
    Data cleaned
    (397884, 8)
    TotalPrice column created
```

```
  InvoiceNo StockCode                          Description  Quantity  \
0    536365    85123A   WHITE HANGING HEART T-LIGHT HOLDER         6
1    536365     71053                  WHITE METAL LANTERN         6
2    536365    84406B       CREAM CUPID HEARTS COAT HANGER         8
3    536365    84029G  KNITTED UNION FLAG HOT WATER BOTTLE         6
4    536365    84029E       RED WOOLLY HOTTIE WHITE HEART.         6

        InvoiceDate  UnitPrice  CustomerID         Country  TotalPrice
0  01-12-2010 08:26       2.55     17850.0  United Kingdom       15.30
1  01-12-2010 08:26       3.39     17850.0  United Kingdom       20.34
2  01-12-2010 08:26       2.75     17850.0  United Kingdom       22.00
3  01-12-2010 08:26       3.39     17850.0  United Kingdom       20.34
4  01-12-2010 08:26       3.39     17850.0  United Kingdom       20.34
Date converted successfully
RFM Table Created
            Recency  Frequency  Monetary
CustomerID
12346.0         326          1  77183.60
12347.0           2        182   4310.00
12348.0          75         31   1797.24
12349.0          19         73   1757.55
12350.0         310         17    334.40
Feature scaling completed
```

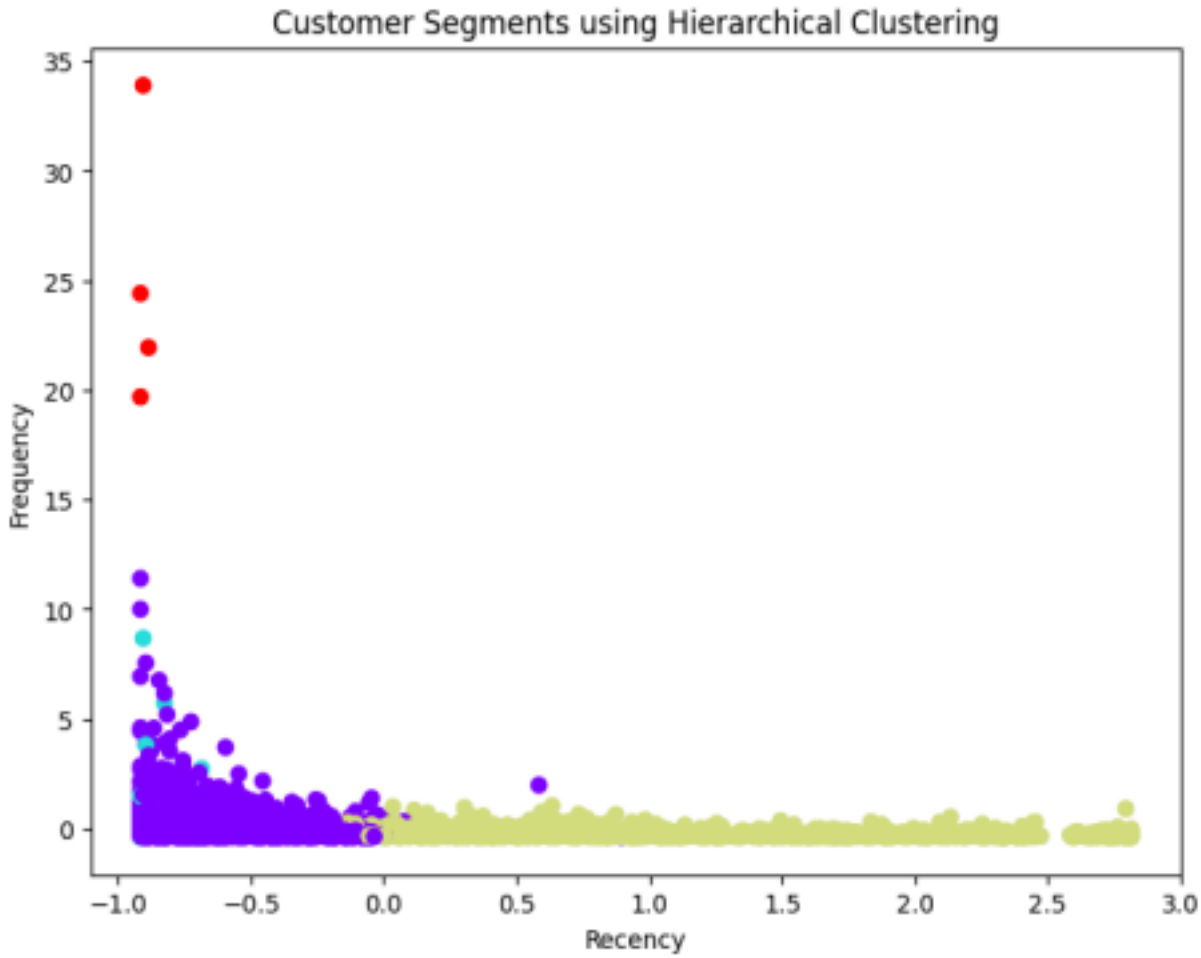


```
KMeans clustering applied
            Recency  Frequency  Monetary  KMeans_Cluster
CustomerID
12346.0         326          1  77183.60               3
12347.0           2        182   4310.00               0
12348.0          75         31   1797.24               1
12349.0          19         73   1757.55               0
12350.0         310         17    334.40               3
```






```
Hierarchical clustering applied
            Recency  Frequency  Monetary  KMeans_Cluster  Hierarchical_Cluster
CustomerID
12346.0         326          1  77183.60               3                     0
12347.0           2        182   4310.00               0                     0
12348.0          75         31   1797.24               1                     0
12349.0          19         73   1757.55               0                     0
12350.0         310         17    334.40               3                     2
```



Cluster Summary

KMeans_Cluster	Recency	Frequency	Monetary	Hierarchical_Cluster
0	20.990788	135.285122	2645.818172	0.001842
1	98.218703	37.719457	773.877083	0.989442
2	4.692308	2565.307692	126118.310000	1.461538
3	272.407005	25.138889	605.842525	1.995169

Results

1. Optimal number of clusters identified: **5**

2. Both K-Means and Hierarchical clustering produced similar segmentation.

3. Customer groups identified:

    - High Income – High Spending
    - High Income – Low Spending
    - Low Income – High Spending
    - Low Income – Low Spending

    - Moderate Income – Moderate Spending

4. Silhouette scores (optional evaluation):

    - K-Means: ~0.55 (approx.)
    - Hierarchical: ~0.53 (approx.)

Conclusion

- K-Means clustering efficiently segmented customers into 5 meaningful groups. ● Hierarchical clustering provided a visual tree structure (dendrogram) for better cluster interpretation.
- K-Means is computationally faster for large datasets.
- Hierarchical clustering provides better interpretability.
- Customer segmentation can help businesses in:
    - Targeted marketing
    - Loyalty programs
    - Pricing strategies
    - Personalized promotions

Thus, clustering techniques successfully grouped customers based on purchasing behavior.